Contrato dos repositories por domÃnio â€” patients, sessions e records

Objetivo
Definir os contratos base dos repositories de domÃnio do Ethos para suportar a migraÃ§Ã£o local-first, comeÃ§ando pelos domÃnios centrais:
- `patients`
- `sessions`
- `records`

Este documento serve como referÃªncia de arquitetura para implementaÃ§Ã£o, testes e refactor incremental do frontend.

PrincÃpios

1. Repository nÃ£o Ã© store de UI
Repository Ã© a interface de acesso ao dado de domÃnio.
Ele nÃ£o deve carregar responsabilidades de navegaÃ§Ã£o, loading visual ou estado de tela.

2. Repository nÃ£o expÃµe detalhes do storage
O contrato nÃ£o deve acoplar a aplicaÃ§Ã£o a IndexedDB, API HTTP ou qualquer backend especÃfico.

3. Repository atende o domÃnio
Os mÃ©todos devem refletir operaÃ§Ãµes relevantes de produto e domÃnio, e nÃ£o apenas CRUD genÃ©rico sem contexto.

4. Local-first por padrÃ£o
Os contracts abaixo assumem que a persistÃªncia local serÃ¡ a fonte primÃ¡ria de verdade.

5. Sync e remoto sÃ£o preocupaÃ§Ãµes secundÃ¡rias
IntegraÃ§Ã£o remota, sync e backup nÃ£o devem poluir o contrato principal do domÃnio nesta fase.

ConvenÃ§Ãµes gerais

Tipos sugeridos
- `Id = string`
- datas sempre em formato consistente e serializÃ¡vel
- entidades persistidas devem conter metadados mÃnimos para evoluÃ§Ã£o futura local-first

Metadados recomendados para entidades persistidas
Sempre que fizer sentido, entidades devem suportar campos como:
- `id`
- `createdAt`
- `updatedAt`
- `deletedAt?`
- `version`
- `localOnly?`
- `syncStatus?`

Esses campos nÃ£o precisam aparecer em todas as telas, mas o contrato deve permitir evoluÃ§Ã£o para isso.

---

1. Patients Repository

Papel do domÃnio
Gerenciar o ciclo de vida do paciente no Ethos.

Inclui:
- cadastro
- atualizaÃ§Ã£o
- consulta
- listagem
- arquivamento ou remoÃ§Ã£o lÃ³gica
- recuperaÃ§Ã£o por identificadores relevantes

Interface sugerida

```ts
export interface Patient {
  id: string;
  fullName: string;
```

```
  email?: string;
  phone?: string;
  birthDate?: string;
  notes?: string;
  status?: "active" | "archived";
  createdAt: string;
  updatedAt: string;
  deletedAt?: string | null;
  version: number;
  syncStatus?: "local" | "pending_sync" | "synced" | "conflict";
}

export interface PatientListFilters {
  query?: string;
  status?: "active" | "archived";
  limit?: number;
  offset?: number;
}

export interface CreatePatientInput {
  fullName: string;
  email?: string;
  phone?: string;
  birthDate?: string;
  notes?: string;
}

export interface UpdatePatientInput {
  fullName?: string;
  email?: string;
  phone?: string;
  birthDate?: string;
  notes?: string;
  status?: "active" | "archived";
}

export interface PatientsRepository {
  getById(id: string): Promise<Patient | null>;
  list(filters?: PatientListFilters): Promise<Patient[]>;
  create(input: CreatePatientInput): Promise<Patient>;
  update(id: string, input: UpdatePatientInput): Promise<Patient>;
  archive(id: string): Promise<void>;
  restore(id: string): Promise<void>;
  remove(id: string): Promise<void>;
  exists(id: string): Promise<boolean>;
}
```

Comportamentos esperados

`getById`
Retorna o paciente ou `null`.

`list`
Lista pacientes por critÃ©rios de busca e estado.

`create`
Cria paciente localmente como nova entidade de domÃnio.

`update`
Atualiza apenas campos permitidos.

`archive`
Arquiva logicamente sem destruir histÃ³rico relacionado.

`restore`
Reativa paciente arquivado.

`remove`
NÃ£o deve apagar fisicamente por padrÃ£o nesta fase, a menos que a polÃtica do domÃnio permita. Preferir remoÃ§Ã£o lÃ³gica.

---

2. Sessions Repository

Papel do domÃnio
Gerenciar sessÃµes clÃnicas vinculadas a pacientes.

Inclui:
- criaÃ§Ã£o
- atualizaÃ§Ã£o
- consulta
- listagem por paciente
- listagem geral
- estado da sessÃ£o
- vÃnculo com prontuÃ¡rio e documentos futuros

Interface sugerida

```ts
export interface Session {
  id: string;
  patientId: string;
  scheduledAt?: string;
  startedAt?: string;
  endedAt?: string;
  status: "scheduled" | "completed" | "cancelled" | "no_show" | "draft";
  summary?: string;
  notes?: string;
  createdAt: string;
  updatedAt: string;
  deletedAt?: string | null;
  version: number;
  syncStatus?: "local" | "pending_sync" | "synced" | "conflict";
}

export interface SessionListFilters {
  patientId?: string;
  status?: Session["status"];
  from?: string;
  to?: string;
  limit?: number;
  offset?: number;
}

export interface CreateSessionInput {
  patientId: string;
  scheduledAt?: string;
  startedAt?: string;
  endedAt?: string;
  status?: Session["status"];
  summary?: string;
  notes?: string;
}

export interface UpdateSessionInput {
  scheduledAt?: string;
  startedAt?: string;
  endedAt?: string;
  status?: Session["status"];
  summary?: string;
  notes?: string;
}

export interface SessionsRepository {
  getById(id: string): Promise<Session | null>;
  list(filters?: SessionListFilters): Promise<Session[]>;
  listByPatient(patientId: string): Promise<Session[]>;
  create(input: CreateSessionInput): Promise<Session>;
  update(id: string, input: UpdateSessionInput): Promise<Session>;
  remove(id: string): Promise<void>;
  exists(id: string): Promise<boolean>;
}
```

Comportamentos esperados

`listByPatient`
Deve ser otimizado como operaÃ§Ã£o central do produto.

`create`
Cria sessÃ£o vinculada a um `patientId` vÃ¡lido.

`update`
NÃ£o deve permitir quebrar integridade do vÃnculo com paciente sem regra explÃcita.

`remove`
Preferir remoÃ§Ã£o lÃ³gica ou polÃtica consistente com auditoria clÃnica futura.

---

3. Records Repository

Papel do domÃnio
Gerenciar o prontuÃ¡rio clÃnico e os registros relacionados Ã sessÃ£o ou ao acompanhamento do paciente.

Aqui â€œrecordsâ€ representa o nÃºcleo do prontuÃ¡rio.

Inclui:
- leitura de registro por id
- leitura por sessÃ£o
- leitura por paciente
- criaÃ§Ã£o de novo registro
- atualizaÃ§Ã£o de conteÃºdo
- versionamento mÃnimo
- histÃ³rico local

Interface sugerida

```ts
export interface ClinicalRecord {
  id: string;
  patientId: string;
  sessionId?: string | null;
  title?: string;
  content: string;
  kind?: "session_note" | "evolution" | "assessment" | "draft" | "general";
  createdAt: string;
  updatedAt: string;
  deletedAt?: string | null;
  version: number;
  syncStatus?: "local" | "pending_sync" | "synced" | "conflict";
}

export interface RecordListFilters {
  patientId?: string;
  sessionId?: string;
  kind?: ClinicalRecord["kind"];
  limit?: number;
  offset?: number;
}

export interface CreateRecordInput {
  patientId: string;
  sessionId?: string | null;
  title?: string;
  content: string;
  kind?: ClinicalRecord["kind"];
}

export interface UpdateRecordInput {
  title?: string;
  content?: string;
  kind?: ClinicalRecord["kind"];
```

```
}

export interface RecordsRepository {
  getById(id: string): Promise<ClinicalRecord | null>;
  list(filters?: RecordListFilters): Promise<ClinicalRecord[]>;
  listByPatient(patientId: string): Promise<ClinicalRecord[]>;
  listBySession(sessionId: string): Promise<ClinicalRecord[]>;
  create(input: CreateRecordInput): Promise<ClinicalRecord>;
  update(id: string, input: UpdateRecordInput): Promise<ClinicalRecord>;
  remove(id: string): Promise<void>;
  exists(id: string): Promise<boolean>;
}
```

Comportamentos esperados

`listByPatient`
Permite reconstruir histÃ³rico clÃnico longitudinal.

`listBySession`
Permite abrir rapidamente o prontuÃ¡rio relacionado Ã sessÃ£o.

`create`
Cria registro clÃnico local com versionamento inicial.

`update`
Deve atualizar `updatedAt` e `version`.

`remove`
Preferir polÃtica compatÃvel com auditoria clÃnica e histÃ³rico, evitando exclusÃ£o fÃsica imediata por padrÃ£o.

---

4. Contratos transversais recomendados

4.1 Unit of work futura
Nesta fase nÃ£o Ã© obrigatÃ³rio, mas a arquitetura deve permitir no futuro operaÃ§Ãµes transacionais entre domÃnios, por exemplo:
- criar sessÃ£o + criar prontuÃ¡rio draft
- arquivar paciente + refletir em entidades relacionadas

4.2 Eventos de domÃnio futuros
Opcionalmente, a camada de services poderÃ¡ emitir eventos como:
- `patient.created`
- `session.completed`
- `record.updated`

Isso nÃ£o precisa fazer parte do contrato do repository agora.

4.3 Erros de domÃnio
Recomenda-se padronizar erros como:
- `NotFoundError`
- `ValidationError`
- `ConflictError`
- `IntegrityError`

---

5. O que nÃ£o deve entrar no contrato agora

NÃ£o incluir agora no contrato principal:
- mÃ©todos de sync remoto
- mÃ©todos de upload/download cloud
- detalhes de backup
- detalhes de criptografia
- estado de loading de UI
- detalhes de analytics

Essas preocupaÃ§Ãµes pertencem a outras camadas.

---

6. EstratÃ©gia de implementaÃ§Ã£o

Fase 1
Implementar contratos e adapters locais para:
- `PatientsRepository`
- `SessionsRepository`

Fase 2
Implementar `RecordsRepository` como base do prontuÃ¡rio local-first.

Fase 3
Adaptar hooks do frontend para consumir services que dependem desses repositories.

Fase 4
Manter adapters remotos separados para migraÃ§Ã£o incremental e compatibilidade temporÃ¡ria.

---

7. CritÃ©rios de sucesso

Este contrato serÃ¡ considerado adequado se:
- permitir implementaÃ§Ã£o local com IndexedDB sem vazar detalhes do storage
- permitir futura camada de sync sem reescrever a interface pÃºblica do domÃnio
- suportar migraÃ§Ã£o incremental de patients, sessions e records
- reduzir acoplamento da UI ao backend remoto

---

8. PrÃ³ximos artefatos recomendados

Depois deste documento, os prÃ³ximos mais Ãºteis sÃ£o:
1. checklist tÃ©cnico da implementaÃ§Ã£o dos repositories locais
2. contrato dos domain services para patients, sessions e records
3. plano de migraÃ§Ã£o de hooks e pÃ¡ginas para consumo local-first
4. schema inicial do banco local para esses trÃªs domÃnios